

PYTHON DICTIONARY

➤ WHAT IS DICTIONARY??

A dictionary in Python is a collection of key:value pairs. Curly brackets "{}" are used for the construction of the dictionary. The key and its value are separated by the colon (:) and each pair is separated by the comma.

Example:

```
details = {'first_name': 'Rohit', 'last_name': 'Kumar', 'emp_id': 200,  
          'bill': 200.2}  
print(details)
```

Output:

```
{'first_name': 'Rohit', 'last_name': 'Kumar', 'emp_id': 200, 'bill': 200.2}
```

➤ ACCESS DICTIONARY ITEMS

- USING KEYS ➔ To access dictionary items, we use the key inside the square brackets.

Example:

```
emp_details = {'first_name': 'Rahul', 'last_name': 'Kumar',
               'emp_id': 200}
print(emp_details['emp_id'])
```

Output:

200

- USING get() Method ⇒ Using the get() method, we do not get any key error if the key does not exist.

Example:

```
emp_details = {'first_name': 'Rahul', 'last_name': 'Kumar', 'emp_id':
               200}
print(emp_details.get('first_name'))
print(emp_details.get('salary'))
```

Output:

Rahul
None

- The items() Method ⇒ The items() method returns the view of tuples, and each tuple contains the key and its value.

Example:

```
numbers = {'1': 'One', '2': 'Two', '3': 'Three'}
print(numbers.items())
```

Output:

```
dict_items([('1', 'One'), ('2', 'Two'), ('3', 'Three')])
```

- The keys() Method ⇒ The keys() method returns the view of keys.

Example:

```
numbers = {'1': 'One', '2': 'Two', '3': 'Three'}
print(numbers.keys())
```

Output:

```
dict_keys(['1', '2', '3'])
```

- The values() Method $\Rightarrow$ The values() method returns the view of values in the dictionary which can be traversed using the for loop.

Example:

```
numbers = {'1': 'One', '2': 'Two', '3': 'Three'}
print(numbers.values())
```

Output:

```
dict_values(['One', 'Two', 'Three'])
```

- for loop $\Rightarrow$ We can use the for loop to loop through the items of the dictionary.

Example:

```
numbers = {'1': 'One', '2': 'Two', '3': 'Three'}
for i in numbers:
    print(i)
```

Output:

```
1
2
3
```

We can use the items(), keys() or values() method if we want items, keys or values in output.

> DICTIONARIES ARE ORDERED

Dictionaries are ordered from Python version 3.7, and before that dictionaries used to be unordered.

> CHANGEABLE

Dictionaries are changeable, means we can change, add or remove items in the dictionary any time.

> DICTIONARY KEYS

Dictionary keys are special because of these properties:->

(1) UNIQUE: The key of the dictionary can not be repeated.

Example:

```
numbers = {'1': 'One', '2': 'Two', '1': 'Three'}  
print(numbers)
```

Output:

```
{'1': 'Three', '2': 'Two'}
```

(2) ONLY MUTABLE ELEMENTS: The key in the dictionary can have only immutable elements like number, string, tuple, etc. Mutable elements like a list or the dictionary itself are not allowed as the key.

Example:

```
numbers = {["Three", "Four"]: ["Three", "Four"]}  
print(numbers)
```


Output:

`TypeError: unhashable type: 'list'`

> MEMBERSHIP TEST =>

Membership operators `in` and `not in` are used to check whether a key is present in the dictionary or not.

```
numbers = {1: "One", 2: "Two", 3: "Three"}
print(1 in numbers)
print(2 not in numbers)
```

Output:

True

False

> NESTED DICTIONARY

A nested dictionary means when we have a dictionary inside another dictionary.

Example:

```
employees = {
    "Rohit": {
        "age": 55,
        "gender": "Male"
    },
    "Steve": {
        "age": 57,
        "gender": "Male"
    }
}
```

```
print(employees["Steve"])
```

```
print (employees["Rohit"]["age"])
```

Output:

```
'age': 57, 'gender': 'Male'}  
55
```

